

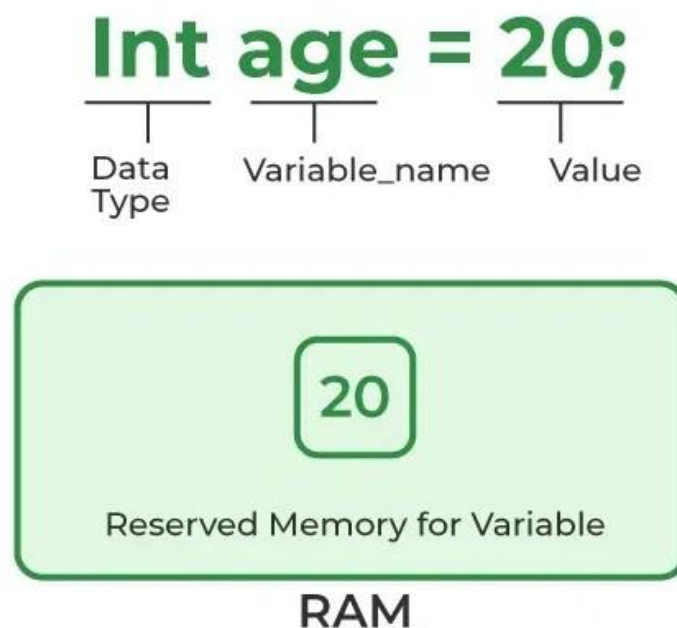
Java

Variables in Java :-

- A variable is a name given to a memory location.
- It is used to store data values in a program.
- In Java, every variable must have a data type.
- Variables can be changed during program execution (unless declared final).

How to Declare Java Variables ?

- It can be perceived with the help of 3 components explained above:



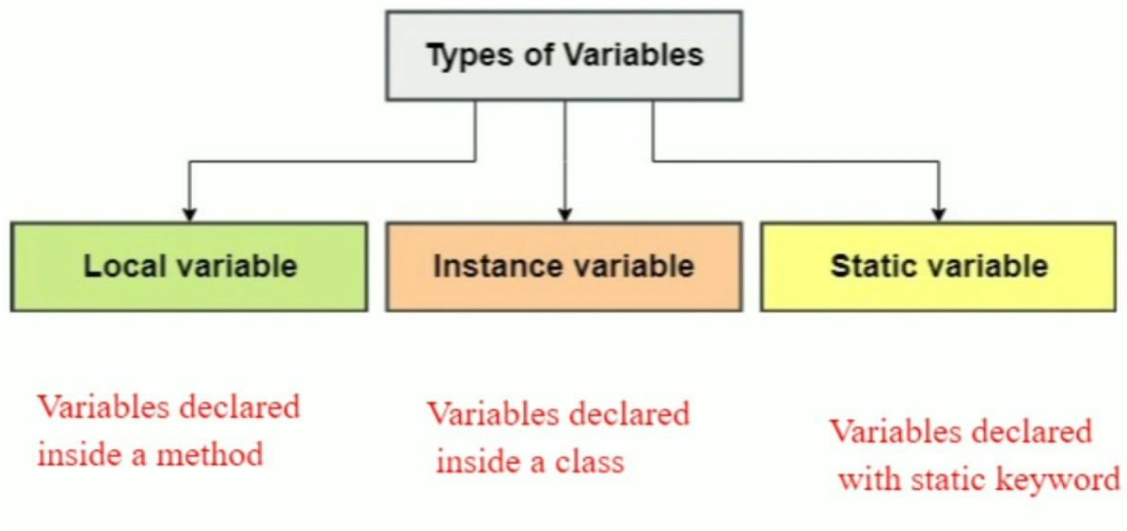
A variable in Java has three components, which are listed below:

- **Data Type:** Defines the kind of data stored (e.g., int, String, float).
- **Variable Name:** A unique identifier following Java naming rules.
- **Value:** The actual data assigned to the variable.

Types of Variables :-

There are different types of variables which are listed as follows:

- 1) Local Variables
- 2) Instance Variables
- 3) Static Variables



1) Local Variables –

- Declared inside a method, block, or constructor.
- Created when the method starts, destroyed when it ends.
- Scope exists only within the block where it is declared.
- Must be initialized before use.

Example –

```
class Demo {  
  
    void show() {  
  
        int x = 10; // Local variable  
  
        System.out.println("Local Variable: " + x);  
  
    }  
  
}
```

2) Instance Variable (Global Variable) –

- Declared in a class, outside methods/constructors/blocks.
- Created when an object is created, destroyed when the object is destroyed.
- Can use access specifiers; default is package-private.
- Initialization is optional (default values depend on type).
- Scope: entire class (except static contexts).

Example –

```
class Student {
```

```
String name; // Instance variable

int age; // Instance variable

}
```

3) Static Variable –

- Declared with static keyword inside a class, outside methods/constructors/blocks.
- Only one copy per class, shared by all objects.
- Created at program start, destroyed when program ends.
- Initialization optional (default values depend on type).
- Accessed via class name (object access allowed but not recommended).

Example –

```
class Student {

    static String school = "ABC School"; // Static variable

}
```

Example Code –

```
package Basic_Concept;

public class variable_types {
    static int n=450; //GlobalVariable/Instance Variable
    static byte age=90; //Static Variable
    static void first()
    {
        int num=60; //Local Variable
        System.out.println("Value of Number is: "+num);
        System.out.println(age);
    }
    void second()
    {
        System.out.println("Value of Number is: "+n);
    }

    public static void main(String[] args) {
        System.out.println(age);
    }

}
```

Output –

90

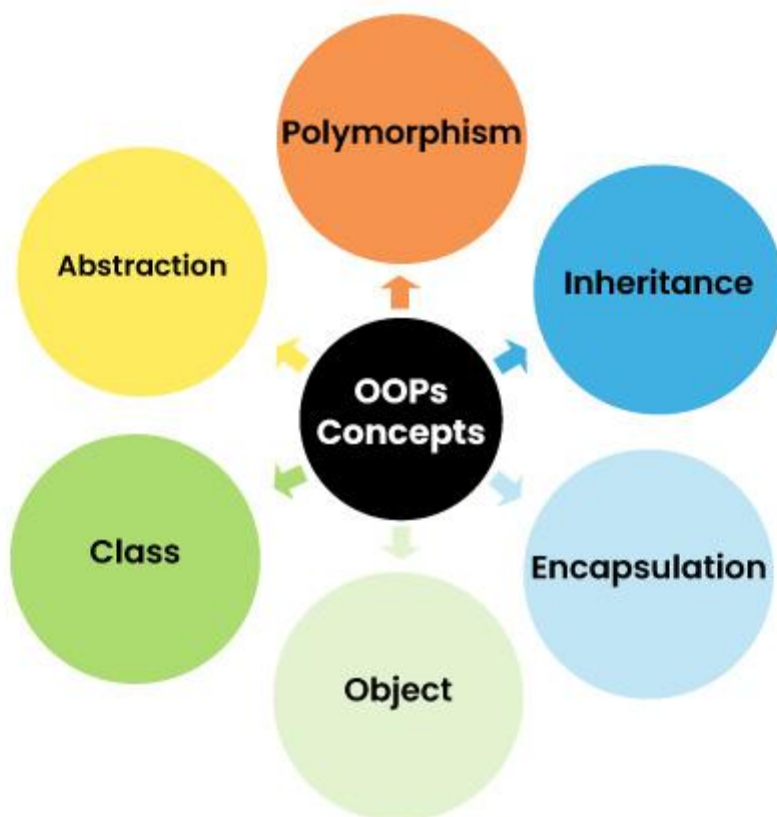
Object Oriented Programming System (OOPS) :-

What is OOPS Concept ?

- OOPS is a Object-Oriented Programming System.
- OOPS is a programming approach that organizes software around objects.
- An object contains data (attributes) and methods (functions/behaviors).
- OOPS makes programs modular, reusable, and easier to maintain.
- OOPS is based on Class, Object, Encapsulation, Inheritance, Polymorphism, and Abstraction.

Why we needs OOPS ?

- OOPS models real-world entities as objects.
- It provides modularity by dividing programs into classes and objects.
- It allows code reusability through inheritance.
- It ensures easy maintainability of code.
- It offers flexibility via polymorphism.
- It provides data security through encapsulation.



1) Class –

- A class is a blueprint or template for creating objects.
- It defines attributes (data members) and methods (functions/behaviors).
- A class does not occupy memory until an object is created.
- It represents a group of similar objects with common properties and behaviors.

Example –

```
class Car {  
  
    String color;  // Attribute  
  
    int speed;    // Attribute  
  
    void drive() { // Method  
  
        System.out.println("Car is driving");  
  
    }  
  
}
```

2) Object –

- An object is an instance of a class.
- It has its own copy of attributes defined in the class.
- Objects can use the methods of the class to perform actions.
- Each object represents a real-world entity with specific values.

Example –

```
Car myCar = new Car(); // Creating an object  
  
myCar.color = "Red"; // Assigning value to attribute  
  
myCar.speed = 120;  
  
myCar.drive(); // Calling method
```

Example Code –

```
package oops;

public class person {
    String name;
    Long phno;

    void walk()
    {
        System.out.println("Walking...");
    }

    void speak()
    {
        System.out.println("Speaking...");
    }

    public static void main(String[] args) {
        //creating of object
        person p1=new person();
        p1.name="Sagar";    //assigning the fields for person 1
        p1.phno=9890824905L;

        person p2=new person();
        p2.name="Rahul";    //assigning the fields for person 2
        p2.phno=8999724839L;

        System.out.println("Person 1 Name:      "+p1.name);
        System.out.println("Person 1 Phone Number: "+p1.phno);
        p1.walk();
        p1.speak();

        System.out.println("Person 2 Name:      "+p2.name);
        System.out.println("Person 2 Phone Number: "+p2.phno);
        p2.walk();
        p2.speak();
    }
}
```

Output –

Person 1 Name: Sagar

Person 1 Phone Number: 9890824905

Walking...

Speaking...

Person 2 Name: Rahul

Person 2 Phone Number: 8999724839

Walking...

Speaking...